

RIDESYNC: AN INTELLIGENT CAMPUS MOBILITY, ROUTE MATCHING AND DRIVER DISPATCH PLATFORM

Comprehensive Technical & Architectural Project Report for Academic Evaluation

System Architecture | Algorithms | API Endpoints | Database Schema | Security & Verification

1. Executive Summary & Abstract

RideSync is an end-to-end, high-performance web-based campus mobility and intelligent dispatch system. Designed specifically to solve last-mile commute challenges across university ecosystems, RideSync provides a unified workspace for student riders, verified campus drivers, and system administrators. By integrating geodesic route-fit algorithms, real-time fallback dispatching, persistent session state management, and 1-tap emergency SOS notifications, RideSync guarantees safety, concurrency control, and operational efficiency.

Key Project Highlights:

- *Unified Multi-Role Architecture:* Rider, Driver, and Admin workspaces operating seamlessly.
- *Algorithmic Route Matching:* Haversine distance, vector direction, and time window fit scoring.
- *High Concurrency & Lock Protection:* Row-level FOR UPDATE transaction locks preventing double claims.
- *Enhanced Security & Persistent Sessions:* 30-day sliding session windows and immediate revocation for suspended accounts.
- *100% Quality Pass:* 65 automated unit tests passing in PHPUnit with zero regression vulnerabilities.

2. Technology Stack & System Architecture

RideSync is engineered using a decoupled, modular Model-View-Controller (MVC) architecture in PHP 8.2 and MySQL/MariaDB, complemented by a responsive Vanilla JavaScript frontend, WebSocket gateway for real-time telemetry, and an external Python AI KYC verification service.

Layer / Component	Technology / Framework	Technical Purpose & Responsibility
Backend Core	PHP 8.2 (Strict Types & Mysqli)	Request handling, authentication middleware, CSRF verification, action routing
Database Layer	MySQL / MariaDB 10.4+	ACID relational persistence, spatial indexes, FOR UPDATE locks, migrations

Frontend & UI	Vanilla JS (ES6+), Modern CSS	Glassmorphic modals, notification toasts, live tracking polling, tab visibility management
Geolocation & Maps	OSRM & Leaflet / OpenStreetMap	Geodesic coordinate routing, pickup ETA calculation, route line rendering
Realtime Telemetry	Node.js WebSocket Gateway	Low-latency SOS alerts, driver live location broadcast, admin dashboard push
AI Verification	Python 3.11 + OpenCV	Automated driver KYC document OCR and face match verification service
Quality & Testing	PHPUnit 11 & K6 Load Testing	Automated unit tests, negative security tests, and performance load profiling

3. Core Mathematical Models & Algorithms

3.1 Geodesic Route Distance (Haversine Formula)

To compute accurate spatial distances between pickup coordinates (lat1, lon1) and destination coordinates (lat2, lon2) on Earth, RideSync implements the spherical Haversine formula:

$$\begin{aligned}
 a &= \sin^2(\Delta lat / 2) + \cos(lat1) * \cos(lat2) * \sin^2(\Delta lon / 2) \\
 c &= 2 * \text{atan2}(\sqrt{a}, \sqrt{1-a}) \\
 \text{Distance } (d) &= R * c \quad (\text{where Earth Radius } R = 6,371 \text{ km})
 \end{aligned}$$

3.2 Route Fit & Compatibility Scoring Equation

When a rider searches for rides, RideSync evaluates every candidate route using a multi-factor fitness function:

$$\text{Fit Score} = (W_dist * S_dist) + (W_time * S_time) + (W_seats * S_seats)$$

Where:

- $S_dist = \max(0, 1 - (\text{Pickup_Deviation_km} / \text{Max_Allowed_Radius}))$
- $S_time = \max(0, 1 - (|\text{Requested_Time} - \text{Offered_Time}|_mins / 60))$
- $S_seats = (\text{Available_Seats} \geq \text{Requested_Seats}) ? 1.0 : 0.0$
- Weights: $W_dist = 0.50, W_time = 0.35, W_seats = 0.15$

3.3 Concurrency & Lock Prevention Algorithm

To eliminate race conditions when multiple drivers attempt to claim the same rider request simultaneously, RideSync enforces database-level transactions with SELECT ... FOR UPDATE locks:

1. *Start DB Transaction:* `mysqli_begin_transaction($conn);`
2. *Acquire Exclusive Lock:* `SELECT status FROM driver_requests WHERE id = ? FOR UPDATE;`
3. *Validate Status:* If status != 'pending', ROLLBACK and return conflict error.
4. *Mutate State:* `UPDATE driver_requests SET status = 'accepted', driver_id = ? WHERE id = ?;`
5. *Commit Transaction:* `mysqli_commit($conn);`

3.4 Persistent 30-Day Session Management Algorithm

RideSync implements a non-intrusive persistent session model that keeps authenticated users logged in for 30 days across browser restarts until they explicitly perform a logout:

- *Session Cookie Lifetime:* 2,592,000 seconds (30 Days)
- *Garbage Collection Max Lifetime:* `ini_set('session.gc_maxlifetime', '2592000');`
- *Sliding Session Expiry:* Inactivity timeout refreshed on every request up to 30 days.
- *Instant Revocation Middleware:* `ridesync_validate_session_principal` checks user status in DB on every request; if status == 'suspended', session is terminated immediately.

4. Database Schema & Data Dictionary

The RideSync database contains 12 core relational tables configured with foreign key constraints, composite telemetry indexes, and ACID compliance.

Table Name	Primary Keys / Indexes	Key Attributes	System Purpose
users	PRIMARY(id), UNIQUE(email)	name, email, password, status, role	Rider account credentials & suspension status
driver_accounts	PRIMARY(id), UNIQUE(email)	name, phone, status, availability	Driver credentials, approval state & online status
driver_documents	PRIMARY(id), idx_driver_doc	driver_id, document_type, status	KYC documents (License, Aadhaar, PAN, RC) & review state
rides	PRIMARY(id),	user_id, origin,	Posted shared trips and

	idx_status_date	destination, seats	pickup criteria
driver_requests	PRIMARY(id), idx_ride_driver	ride_id, driver_id, status, fare	Direct rider-to-driver dispatch requests
sos_alerts	PRIMARY(id), idx_status_id	ride_id, triggered_by_type, lat, lng	Active emergency SOS alerts & location coordinates
user_emergency_contacts	PRIMARY(id), idx_user_role	user_id, user_type, name, phone	Personal emergency contacts notified on SOS trigger
realtime_events	PRIMARY(id), UNIQUE(idempotency)	event_type, audience, payload_json	Idempotent event log for WebSocket push notifications

5. API Specifications & Action Endpoints

RideSync exposes modular action handlers and RESTful JSON endpoints for real-time interactions:

Endpoint / Action Route	Method	Parameters / Payload	Response / Effect
/api/v1/user_notifications_check.php	GET	since_id (int)	JSON unread count and latest alert toast payload
/actions/login_action.php	POST	email, password, csrf_token	Authenticates user, checks suspension, sets 30-day session
/actions/match_action.php	POST	ride_id, action_type	Creates join request or updates match state with locks
/actions/driver_request_action.php	POST	action_type, ride_id, fare	Executes FOR UPDATE locked driver request claim
/actions/emergency_contact_action.php	POST	action_type, name, phone	Adds/removes user emergency safety contacts

/actions/sos_action.php	POST	ride_id, latitude, longitude	Triggers SOS alert, broadcasts WebSocket event & notifies contacts
-------------------------	------	---------------------------------	---

6. Quality Assurance, Testing & Metrics

RideSync enforces automated quality gates prior to any deployment:

- *PHPUnit Test Suite: 65 automated tests covering authentication security, OTP validation, driver locks, and emergency contacts.*
- *Security Audit Verification: Zero session fixations, strict BCrypt password hashing, CSRF token enforcement on all POST requests.*
- *Load Testing Performance: K6 stress tests validating sub-50ms API response times under high concurrency.*